# DAG Pipeline Declared by `pipeline.yaml`

## DAG Pipeline Declared by `pipeline.yaml` I

Single-project pipelines read `infrastructure/core/pipeline/pipeline.yaml`. `scripts/runner/execute_pipeline.py` expands the declarative DAG, applies tag filters (`--mode core` skips `llm` stages), checkpoints between nodes, then dispatches the declared `scripts/pipeline/stage_NN_*.py` scripts or builtin methods (`_run_clean_outputs`).

The YAML declares **17 stages**. Tag filtering selects **10 stages for a default full run** and **8 for core mode**. The overview below covers the default full selection: core stages plus live LLM stages. Telemetry settings are configuration metadata, not additional DAG nodes.

## DAG Pipeline Declared by `pipeline.yaml` I

1. **Clean Output Directories** — wipes prior `projects/<name>/output/` + delivered `output/<name>/` paths so stale PDFs cannot satisfy validation.
2. **Environment Setup** (`scripts/pipeline/stage_00_setup.py`) — Python/uv probing, toolchain discovery, scaffolding directories, `PYTHONPATH` wiring.
3. **Infrastructure Tests** (`scripts/pipeline/stage_01_test.py`, `--infra-only`, `--infra-scope pipeline-smoke`) — focused real infrastructure contracts for the project pipeline. The full infrastructure suite and its configured $\geq$60% coverage floor remain separate verification.
4. **Project Tests** (`scripts/pipeline/stage_01_test.py --project-only`) — isolated per-project suites with each project's declared floor ($\geq$90% for this exemplar).
5. **Project Analysis** (`scripts/pipeline/stage_02_analysis.py`) — lexicographically ordered `projects/<name>/scripts/*.py`, each a thin orchestrator (`src/` does real work).
6. **PDF Rendering** (`scripts/pipeline/stage_03_render.py`) — Pandoc $\rightarrow$ XeLaTeX loop, bibliography assembly, injected variables from Stage 02 artefacts.
7. **Output Validation** (`scripts/pipeline/stage_04_validate.py`) — PDF structure, manifests, Markdown hygiene.

# DAG Pipeline Declared by `pipeline.yaml` II

8. **LLM Manuscript Review** (`scripts/pipeline/stage_06_llm_review.py --reviews-only`; `tags: llm`) — executive + quality critiques via local Ollama; `allow_skip: true`.
9. **LLM Translations** (`scripts/pipeline/stage_06_llm_review.py --translations-only`; tags `llm`, same dependency edges) — multilingual abstract expansion.
10. **Copy Outputs** (`scripts/pipeline/stage_05_copy.py`) — reproducible snapshots into canonical `output/<project>/`.

The two LLM nodes share one script module with orthogonal CLI switches and depend on output validation. Full mode includes them; core mode excludes them. When a stage permits a skip, it must retain a fresh policy-bound reason receipt; unavailable required live acceptance remains blocked.

**Executive reporting** (`scripts/pipeline/stage_07_executive_report.py`) is a separate multi-project reporting operation, not an additional YAML node inside the single-project executor. `execute_multi_project.py` writes aggregate summary reports after project execution.

## DAG Pipeline Declared by `pipeline.yaml` I

Topological order therefore differs slightly from declaration order (e.g., copy executes after validation even though stage `stage_05_copy.py` precedes `stage_06_llm_review.py` lexically).

**Infrastructure and project tests.** Separate invocations isolate the focused infrastructure smoke contract from each project's nonempty test suite and project-local coverage gate. Required test or coverage failures and unsuccessful runners block pipeline completion; configured failure tolerances cannot turn a nonzero child test result into success. The full infrastructure coverage floor ($\geq$60%) is verified separately. An explicitly skippable stage may accept exit 2 only with a fresh reason receipt bound to the current run, stage, command, and plan. That recorded skip does not satisfy required live acceptance.

**Stage 02 illustration.** The analysis stage is deliberately concrete rather than a hypothetical diagram factory: each canonical project ships real behaviour at this node. `template_autoresearch_project` runs readiness validation; `template_search_project` merges remote literature JSON, generates scripted figures (`y_generate_search_figures.py`), and writes manifests; `template_code_project` emits optimization plots; and `template_prose_project` triggers structural validation scaffolding. The pipeline shape is identical across all four—only the Stage 02 payload differs—which is exactly what lets one orchestrator serve heterogeneous research domains. The live public roster is generated in `active_projects.md`.

## run.sh |

Thin wrapper invoking `python -m infrastructure.orchestration`. Offers:

## run.sh |

- per-project staged execution,
- chained digits (234 shorthand),
- multi-project grid (a–d presets),
- graceful quit / resume parity with `scripts/README.md`.

Selecting `d` **alone** after a passing multi-project run exits immediately once summaries print—avoiding repetitive menu redraw.

## `secure_run.sh`

Executes Python `secure` path: standard pipeline artefact reproduction **then** invokes `run_secure_pipeline` for steganographic PDF hardening (`infrastructure.steganography`). Original PDFs stay immutable; hardened companions carry QR overlays plus hash manifests sidecars.